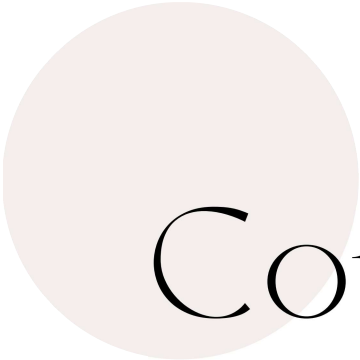



Course registration

JAVA CASE STUDY

OVERVIEW OF COURSEREGISTRATION SYSTEM

- Console-based Java application
- Manages students, courses, and enrollment
- Real-world academic registration simulation
- Implements core Java OOP principles

PROJECT OBJECTIVES

- Enable adding students and courses
- Allow student enrollment & course drop
- Maintain course capacity rules
- Provide real-time console menu operations
- Demonstrate data structures & OOP concepts

TECHNOLOGIES USED

- Java (Core Java)
- Collections Framework
- Scanner for user input
- HashMap & HashSet
- OOP Concepts (Classes, Objects, Encapsulation)

SYSTEM ARCHITECTURE

- Main class creates system & loads demo data
- Student class → stores student info & courses
- Course class → course details & student list
- RegistrationSystem class → handles menu & logic
- Maps store data in memory

FUNCTIONALITIES

- Add Student
- Add Course
- Enroll Student
- Drop Student
- List Students
- List Courses
- View Student's Courses
- View Course's Students

ADVANTAGES

- Simple & user-friendly
- Efficient data access (HashMap)
- Scalable for additional features
- Real-world academic system model

CHALLENGES FACED

- Ensuring data consistency on enroll/drop
- Input validation & handling wrong inputs
- Maintaining two-way data sync (student & course lists)

CONCLUSION

- Successfully built a working registration system
- Demonstrated OOP, Collections, and Java console I/O
- Ready for real-world extension

Program

COURSE REGISTRATION APP

```
import java.util.*;

public class UniversityApp {
    public static void main(String[] args) {
        RegistrationSystem sys = new RegistrationSystem();
        seedDemoData(sys);
        sys.runConsole();
    }

    private static void seedDemoData(RegistrationSystem sys) {
        sys.addStudent(101, "Alice");
        sys.addStudent(102, "Bob");
        sys.addStudent(103, "Charlie");

        sys.addCourse("CSE101", "Intro to CS", 2);
        sys.addCourse("CSE102", "Data Structures", 3);
        sys.addCourse("MAT101", "Calculus I", 2);
    }
}

/* ----- Domain classes ----- */

class Student {
    private final int id;
    private final String name;
    private final Set<String> enrolledCourseIds = new HashSet<>();

    public Student(int id, String name) {
        this.id = id;
        this.name = name.trim();
    }

    public int getId() { return id; }
    public String getName() { return name; }
```

```

    public int getId() { return id; }
    public String getName() { return name; }

    public Set<String> getEnrolledCourseIds() {
        return Collections.unmodifiableSet(enrolledCourseIds);
    }

    boolean enroll(String courseId) {
        return enrolledCourseIds.add(courseId);
    }

    boolean drop(String courseId) {
        return enrolledCourseIds.remove(courseId);
    }

    @Override
    public String toString() {
        return id + " - " + name;
    }
}

class Course {
    private final String id;
    private final String title;
    private final int capacity;
    private final Set<Integer> enrolledStudentIds = new HashSet<>();

    public Course(String id, String title, int capacity) {
        this.id = id.trim().toUpperCase();
        this.title = title.trim();
        this.capacity = Math.max(1, capacity);
    }

    public String getId() { return id; }
    public String getTitle() { return title; }

```

```
public String getTitle() { return title; }
public int getCapacity() { return capacity; }

public Set<Integer> getEnrolledStudentIds() {
    return Collections.unmodifiableSet(enrolledStudentIds);
}

boolean hasSpace() {
    return enrolledStudentIds.size() < capacity;
}

boolean enroll(int studentId) {
    if (!hasSpace()) return false;
    return enrolledStudentIds.add(studentId);
}

boolean drop(int studentId) {
    return enrolledStudentIds.remove(studentId);
}

@Override
public String toString() {
    return id + " - " + title +
        " (Capacity: " + capacity +
        ", Enrolled: " + enrolledStudentIds.size() + ")";
}

}

/* ----- System / Controller ----- */

class RegistrationSystem {
    private final Map<Integer, Student> students = new HashMap<>();
    private final Map<String, Course> courses = new HashMap<>();
    private final Scanner scanner = new Scanner(System.in);
}
```

```
void runConsole() {
    while (true) {
        printMenu();
        String choice = scanner.nextLine().trim();
        switch (choice) {
            case "1": cmdAddStudent(); break;
            case "2": cmdAddCourse(); break;
            case "3": cmdEnroll(); break;
            case "4": cmdDrop(); break;
            case "5": cmdListStudents(); break;
            case "6": cmdListCourses(); break;
            case "7": cmdViewStudentCourses(); break;
            case "8": cmdViewCourseStudents(); break;
            case "9": System.out.println("Exiting."); return;
            default: System.out.println("Invalid option. Try again.");
        }
    }
}

private void printMenu() {
    System.out.println("\n=== University Course Registration ===");
    System.out.println("1. Add Student");
    System.out.println("2. Add Course");
    System.out.println("3. Enroll Student in Course");
    System.out.println("4. Drop Student from Course");
    System.out.println("5. List Students");
    System.out.println("6. List Courses");
    System.out.println("7. View Student's Courses");
    System.out.println("8. View Course's Students");
    System.out.println("9. Exit");
    System.out.print("Choose an option: ");
}
```

```
private void cmdAddStudent() {
```

```

private void cmdAddStudent() {
    try {
        System.out.print("Student ID (integer): ");
        int id = Integer.parseInt(scanner.nextLine().trim());
        System.out.print("Student name: ");
        String name = scanner.nextLine().trim();
        boolean ok = addStudent(id, name);
        System.out.println(ok ? "Student added." : "Student ID already exists.");
    } catch (NumberFormatException e) {
        System.out.println("Invalid ID. Must be an integer.");
    }
}

boolean addStudent(int id, String name) {
    if (students.containsKey(id)) return false;
    students.put(id, new Student(id, name));
    return true;
}

private void cmdAddCourse() {
    System.out.print("Course ID (e.g. CSE101): ");
    String cid = scanner.nextLine().trim().toUpperCase();
    System.out.print("Course title: ");
    String title = scanner.nextLine().trim();
    try {
        System.out.print("Capacity (integer): ");
        int cap = Integer.parseInt(scanner.nextLine().trim());
        boolean ok = addCourse(cid, title, cap);
        System.out.println(ok ? "Course added." : "Course ID already exists.");
    } catch (NumberFormatException e) {
        System.out.println("Invalid capacity. Must be an integer.");
    }
}

```

```

boolean addCourse(String id, String title, int capacity) {

```

```
boolean addCourse(String id, String title, int capacity) {
    id = id.toUpperCase();
    if (courses.containsKey(id)) return false;
    courses.put(id, new Course(id, title, capacity));
    return true;
}

private void cmdEnroll() {
    try {
        System.out.print("Student ID: ");
        int sid = Integer.parseInt(scanner.nextLine().trim());
        System.out.print("Course ID: ");
        String cid = scanner.nextLine().trim().toUpperCase();
        RegistrationResult res = enrollStudentInCourse(sid, cid);
        System.out.println(res.message);
    } catch (NumberFormatException e) {
        System.out.println("Invalid student ID.");
    }
}

private RegistrationResult enrollStudentInCourse(int studentId, String courseId) {
    Student s = students.get(studentId);
    if (s == null) return new RegistrationResult(false, "Student not found.");

    Course c = courses.get(courseId);
    if (c == null) return new RegistrationResult(false, "Course not found.");

    if (s.getEnrolledCourseIds().contains(courseId))
        return new RegistrationResult(false, "Student already enrolled.");

    if (!c.hasSpace())
        return new RegistrationResult(false, "Course is full.");

    boolean cEnrolled = c.enroll(studentId);
    boolean sEnrolled = s.enroll(courseId);
}
```

```
        boolean sEnrolled = s.enroll(courseId);
        if (cEnrolled && sEnrolled) return new RegistrationResult(true, "Enrollment successful.");

        if (cEnrolled) c.drop(studentId);
        if (sEnrolled) s.drop(courseId);
        return new RegistrationResult(false, "Internal error.");
    }

    private void cmdDrop() {
        try {
            System.out.print("Student ID: ");
            int sid = Integer.parseInt(scanner.nextLine().trim());
            System.out.print("Course ID: ");
            String cid = scanner.nextLine().trim().toUpperCase();
            RegistrationResult res = dropStudentFromCourse(sid, cid);
            System.out.println(res.message);
        } catch (NumberFormatException e) {
            System.out.println("Invalid student ID.");
        }
    }

    private RegistrationResult dropStudentFromCourse(int studentId, String courseId) {
        Student s = students.get(studentId);
        if (s == null) return new RegistrationResult(false, "Student not found.");

        Course c = courses.get(courseId);
        if (c == null) return new RegistrationResult(false, "Course not found.");

        if (!s.getEnrolledCourseIds().contains(courseId))
            return new RegistrationResult(false, "Student not enrolled.");

        boolean cDropped = c.drop(studentId);
        boolean sDropped = s.drop(courseId);
        if (cDropped && sDropped) return new RegistrationResult(true, "Dropped successfully.");
    }
}
```

```

    if (cDropped) c.enroll(studentId);
    if (sDropped) s.enroll(courseId);
    return new RegistrationResult(false, "Internal error.");
}

private void cmdListStudents() {
    if (students.isEmpty()) {
        System.out.println("No students added.");
        return;
    }
    System.out.println("Students:");
    students.values().stream()
        .sorted(Comparator.comparingInt(Student::getId))
        .forEach(s -> System.out.println(" " + s));
}

private void cmdListCourses() {
    if (courses.isEmpty()) {
        System.out.println("No courses added.");
        return;
    }
    System.out.println("Courses:");
    courses.values().stream()
        .sorted(Comparator.comparing(Course::getId))
        .forEach(c -> System.out.println(" " + c));
}

private void cmdViewStudentCourses() {
    try {
        System.out.print("Student ID: ");
        int sid = Integer.parseInt(scanner.nextLine().trim());
        Student s = students.get(sid);
        if (s == null) {
            System.out.println("Student not found.");
        }
    }
}

```

Edit View



```

        System.out.println("Student not found.");
        return;
    }
    Set<String> ecs = s.getEnrolledCourseIds();
    if (ecs.isEmpty()) {
        System.out.println(s + " is not enrolled in any courses.");
        return;
    }
    System.out.println(s + " is enrolled in:");
    ecs.forEach(cid -> {
        Course c = courses.get(cid);
        String title = (c == null) ? "(course removed)" : c.getTitle();
        System.out.println("  " + cid + " - " + title);
    });
} catch (NumberFormatException e) {
    System.out.println("Invalid student ID.");
}
}

private void cmdViewCourseStudents() {
    System.out.print("Course ID: ");
    String cid = scanner.nextLine().trim().toUpperCase();
    Course c = courses.get(cid);
    if (c == null) {
        System.out.println("Course not found.");
        return;
    }
    Set<Integer> sids = c.getEnrolledStudentIds();
    if (sids.isEmpty()) {
        System.out.println("No students enrolled in " + c.getId());
        return;
    }
    System.out.println("Students in " + c.getId() + " - " + c.getTitle() + ":");
    sids.stream().sorted().forEach(id -> {
        Student s = students.get(id);

```

```

Edit View
    system.out.println("Invalid student ID. ");
}
}

private void cmdViewCourseStudents() {
    System.out.print("Course ID: ");
    String cid = scanner.nextLine().trim().toUpperCase();
    Course c = courses.get(cid);
    if (c == null) {
        System.out.println("Course not found.");
        return;
    }
    Set<Integer> sids = c.getEnrolledStudentIds();
    if (sids.isEmpty()) {
        System.out.println("No students enrolled in " + c.getId());
        return;
    }
    System.out.println("Students in " + c.getId() + " - " + c.getTitle() + ":");
    sids.stream().sorted().forEach(id -> {
        Student s = students.get(id);
        System.out.println("  " + (s == null ? (id + " (removed)") : s));
    });
}

private static class RegistrationResult {
    final boolean success;
    final String message;

    RegistrationResult(boolean success, String message) {
        this.success = success;
        this.message = message;
    }
}
}

```

(c) Microsoft Corporation. All rights reserved.

C:\Users\harsh\Downloads\Code>javac UniversityApp.java

C:\Users\harsh\Downloads\Code>java UniversityApp.java

=== University Course Registration ===

1. Add Student
2. Add Course
3. Enroll Student in Course
4. Drop Student from Course
5. List Students
6. List Courses
7. View Student's Courses
8. View Course's Students
9. Exit

Choose an option: 1

Student ID (integer): 2020

Student name: medha

Student added.

=== University Course Registration ===

1. Add Student
2. Add Course
3. Enroll Student in Course
4. Drop Student from Course
5. List Students
6. List Courses
7. View Student's Courses
8. View Course's Students
9. Exit

Choose an option: 1

```
Student ID (integer): 2021
Student name: harshmitha
Student added.
```

```
=== University Course Registration ===
```

1. Add Student
2. Add Course
3. Enroll Student in Course
4. Drop Student from Course
5. List Students
6. List Courses
7. View Student's Courses
8. View Course's Students
9. Exit

```
Choose an option: 1
```

```
Student ID (integer): 2022
Student name: veda
Student added.
```

```
=== University Course Registration ===
```

1. Add Student
2. Add Course
3. Enroll Student in Course
4. Drop Student from Course
5. List Students
6. List Courses
7. View Student's Courses
8. View Course's Students
9. Exit

```
Choose an option: 1
```

```
Student ID (integer): 2023
Student name: yuktha
Student added.
```

=== University Course Registration ===

1. Add Student
2. Add Course
3. Enroll Student in Course
4. Drop Student from Course
5. List Students
6. List Courses
7. View Student's Courses
8. View Course's Students
9. Exit

Choose an option: 2

Course ID (e.g. CSE101): maths

Course title: maths

Capacity (integer): 3

Course added.

=== University Course Registration ===

1. Add Student
2. Add Course
3. Enroll Student in Course
4. Drop Student from Course
5. List Students
6. List Courses
7. View Student's Courses
8. View Course's Students
9. Exit

Choose an option: 2

Course ID (e.g. CSE101): physics

Course title: physics

Capacity (integer): 3

Course added.

=== University Course Registration ===

1. Add Student
2. Add Course
3. Enroll Student in Course
4. Drop Student from Course
5. List Students
6. List Courses
7. View Student's Courses
8. View Course's Students
9. Exit

Choose an option: 3

Student ID: 2020

Course ID: maths

Enrollment successful.

=== University Course Registration ===

1. Add Student
2. Add Course
3. Enroll Student in Course
4. Drop Student from Course
5. List Students
6. List Courses
7. View Student's Courses
8. View Course's Students
9. Exit

Choose an option: 3

Student ID: 2021

Course ID: maths

Enrollment successful.

=== University Course Registration ===

1. Add Student

=== University Course Registration ===

1. Add Student
2. Add Course
3. Enroll Student in Course
4. Drop Student from Course
5. List Students
6. List Courses
7. View Student's Courses
8. View Course's Students
9. Exit

Choose an option: 3

Student ID: 2022

Course ID: maths

Enrollment successful.

=== University Course Registration ===

1. Add Student
2. Add Course
3. Enroll Student in Course
4. Drop Student from Course
5. List Students
6. List Courses
7. View Student's Courses
8. View Course's Students
9. Exit

Choose an option: 3

Student ID: maths

Invalid student ID.

=== University Course Registration ===

1. Add Student
2. Add Course

3. Enroll Student in Course
4. Drop Student from Course
5. List Students
6. List Courses
7. View Student's Courses
8. View Course's Students
9. Exit

Choose an option: 3

Student ID: 2023

Course ID: maths

Course is full.

=== University Course Registration ===

1. Add Student
2. Add Course
3. Enroll Student in Course
4. Drop Student from Course
5. List Students
6. List Courses
7. View Student's Courses
8. View Course's Students
9. Exit

Choose an option: 5

Students:

101 - Alice

102 - Bob

103 - Charlie

2020 - medha

2021 - harshmitha

2022 - veda

2023 - yuktha

- 1. Add Student
- 2. Add Course
- 3. Enroll Student in Course
- 4. Drop Student from Course
- 5. List Students
- 6. List Courses
- 7. View Student's Courses
- 8. View Course's Students
- 9. Exit

Choose an option: 6

Courses:
CSE101 - Intro to CS (Capacity: 2, Enrolled: 0)
CSE102 - Data Structures (Capacity: 3, Enrolled: 0)
MAT101 - Calculus I (Capacity: 2, Enrolled: 0)
MATHS - maths (Capacity: 3, Enrolled: 3)
PHYSICS - physics (Capacity: 3, Enrolled: 0)

=== University Course Registration ===

- 1. Add Student
- 2. Add Course
- 3. Enroll Student in Course
- 4. Drop Student from Course
- 5. List Students
- 6. List Courses
- 7. View Student's Courses
- 8. View Course's Students
- 9. Exit

Choose an option: 7

Student ID: 2020
2020 - medha is enrolled in:
MATHS - maths

=== University Course Registration ===

2020 - medha is enrolled in:
MATHS - maths

=== University Course Registration ===

1. Add Student
2. Add Course
3. Enroll Student in Course
4. Drop Student from Course
5. List Students
6. List Courses
7. View Student's Courses
8. View Course's Students
9. Exit

Choose an option: 8

Course ID: maths

Students in MATHS - maths:

2020 - medha
2021 - harshmitha
2022 - veda

=== University Course Registration ===

1. Add Student
2. Add Course
3. Enroll Student in Course
4. Drop Student from Course
5. List Students
6. List Courses
7. View Student's Courses
8. View Course's Students
9. Exit

Choose an option: 9

Exiting.